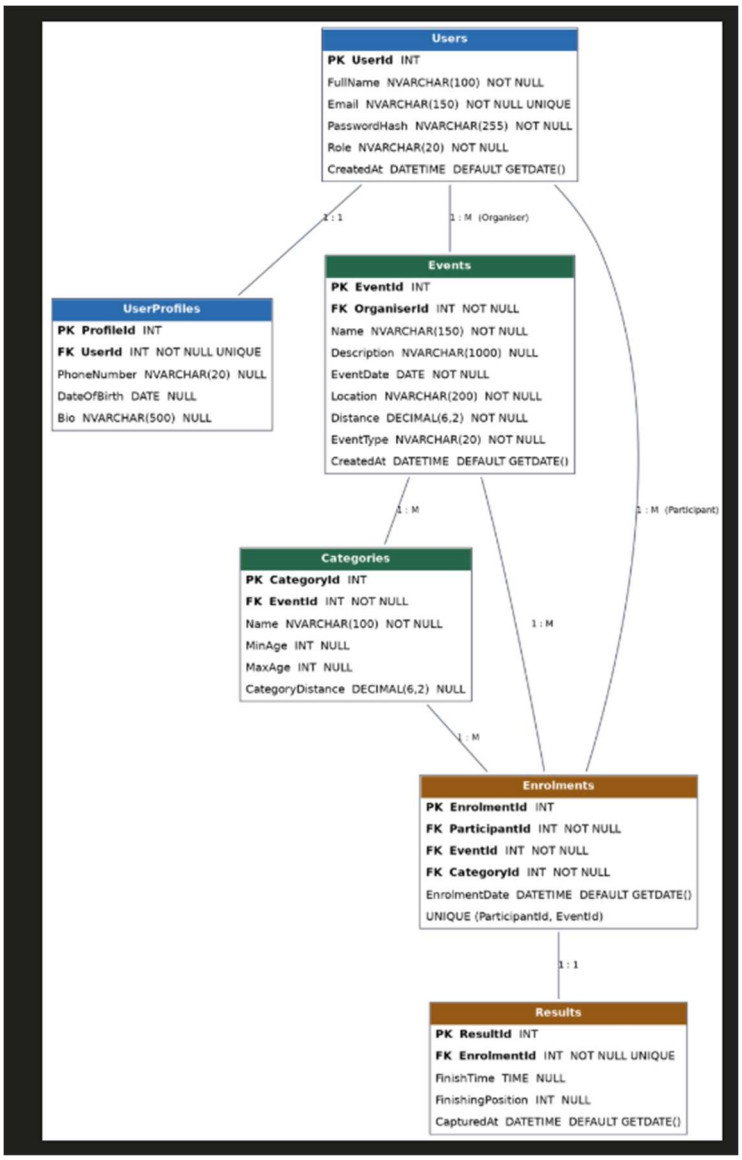
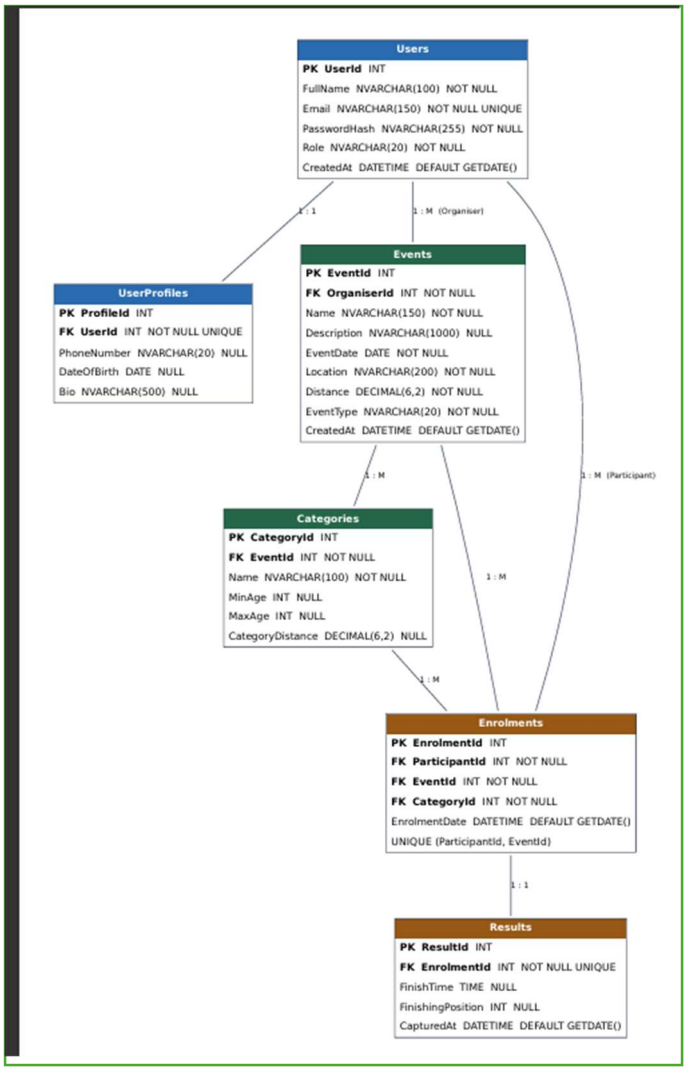
PROGRAMMING 2B (PROG6212)

ST10475585 MUDAU OMPHULUSA IMANI

POE PART 1


Users
PK UserId INT
FullName NVARCHAR(100) NOT NULL
Email NVARCHAR(150) NOT NULL UNIQUE
PasswordHash NVARCHAR(255) NOT NULL
Role NVARCHAR(20) NOT NULL
CreatedAt DATETIME DEFAULT GETDATE()
1 : 1
1 : M (Organiser)
UserProfiles
PK ProfileId INT
FK UserId INT NOT NULL UNIQUE
PhoneNumber NVARCHAR(20) NULL
DateOfBirth DATE NULL
Bio NVARCHAR(500) NULL
Events
PK EventId INT
FK OrganiserId INT NOT NULL
Name NVARCHAR(150) NOT NULL
Description NVARCHAR(1000) NULL
EventDate DATE NOT NULL
Location NVARCHAR(200) NOT NULL
Distance DECIMAL(6,2) NOT NULL
EventType NVARCHAR(20) NOT NULL
CreatedAt DATETIME DEFAULT GETDATE()
1 : M
1 : M (Participant)
Categories
PK CategoryId INT
FK EventId INT NOT NULL
Name NVARCHAR(100) NOT NULL
MinAge INT NULL
MaxAge INT NULL
CategoryDistance DECIMAL(6,2) NULL
1 : M
1 : M
Enrolments
PK EnrolmentId INT
FK ParticipantId INT NOT NULL
FK EventId INT NOT NULL
FK CategoryId INT NOT NULL
EnrolmentDate DATETIME DEFAULT GETDATE()
UNIQUE (ParticipantId, EventId)
1 : 1
Results
PK ResultId INT
FK EnrolmentId INT NOT NULL UNIQUE
FinishTime TIME NULL
FinishingPosition INT NULL
CapturedAt DATETIME DEFAULT GETDATE()


Users
PK UserId INT
FullName NVARCHAR(100) NOT NULL
Email NVARCHAR(150) NOT NULL UNIQUE
PasswordHash NVARCHAR(255) NOT NULL
Role NVARCHAR(20) NOT NULL
CreatedAt DATETIME DEFAULT GETDATE()
1 : 1
1 : M (Organiser)
UserProfiles
PK ProfileId INT
FK UserId INT NOT NULL UNIQUE
PhoneNumber NVARCHAR(20) NULL
DateOfBirth DATE NULL
Bio NVARCHAR(500) NULL
Events
PK EventId INT
FK OrganiserId INT NOT NULL
Name NVARCHAR(150) NOT NULL
Description NVARCHAR(1000) NULL
EventDate DATE NOT NULL
Location NVARCHAR(200) NOT NULL
Distance DECIMAL(6,2) NOT NULL
EventType NVARCHAR(20) NOT NULL
CreatedAt DATETIME DEFAULT GETDATE()
1 : M
1 : M (Participant)
Categories
PK CategoryId INT
FK EventId INT NOT NULL
Name NVARCHAR(100) NOT NULL
MinAge INT NULL
MaxAge INT NULL
CategoryDistance DECIMAL(6,2) NULL
1 : M
1 : M
Enrolments
PK EnrolmentId INT
FK ParticipantId INT NOT NULL
FK EventId INT NOT NULL
FK CategoryId INT NOT NULL
EnrolmentDate DATETIME DEFAULT GETDATE()
UNIQUE (ParticipantId, EventId)
1 : 1
Results
PK ResultId INT
FK EnrolmentId INT NOT NULL UNIQUE
FinishTime TIME NULL
FinishingPosition INT NULL
CapturedAt DATETIME DEFAULT GETDATE()

Sql

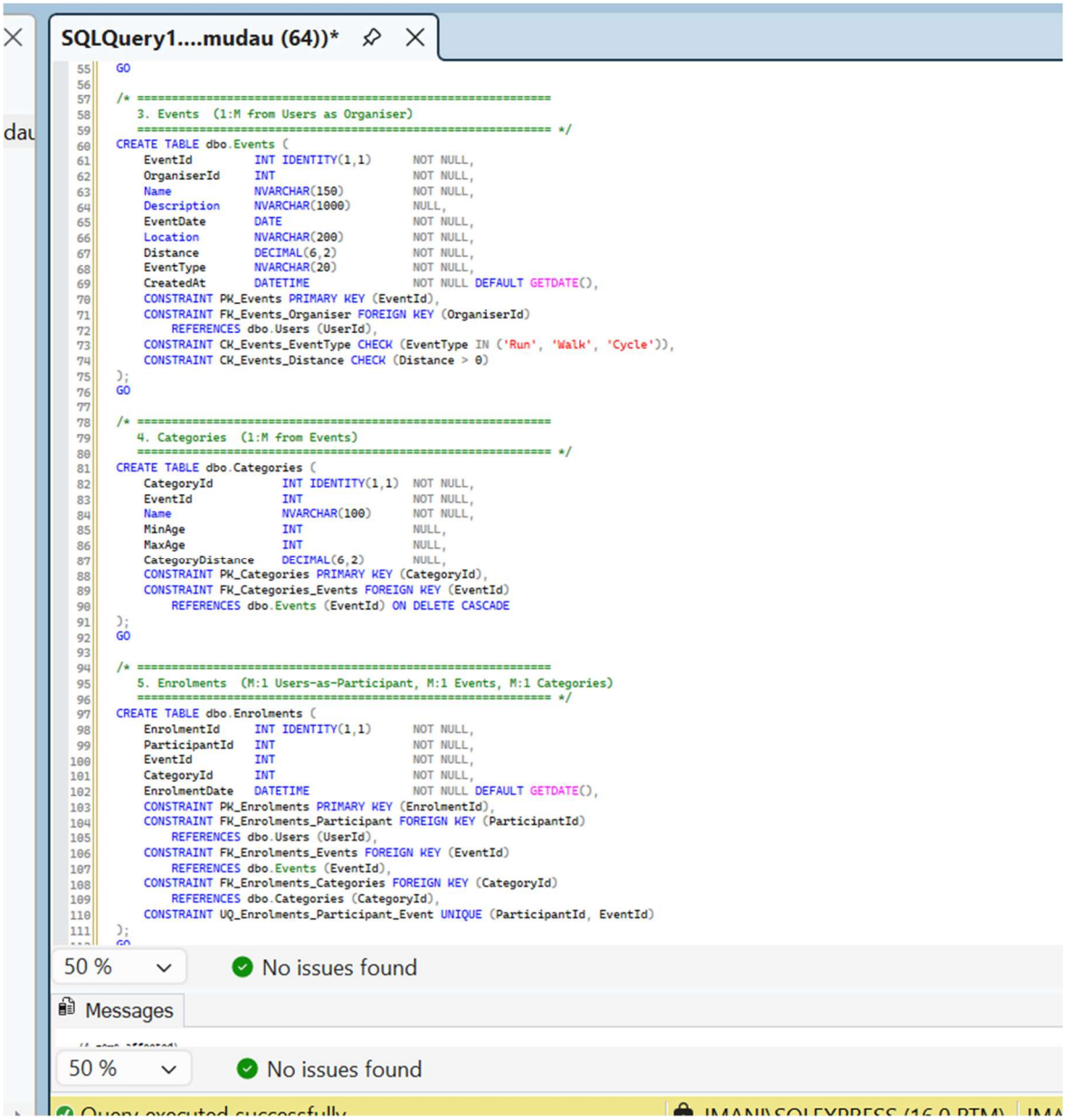
```sql
/* ============================================================
   RaceDay Database Schema
   SQL Server Management Studio (SSMS)
   Matches erd.png exactly - 6 entities, all PKs/FKs/constraints
   ============================================================ */

IF DB_ID('RaceDayDB') IS NULL
BEGIN
    CREATE DATABASE RaceDayDB;
END
GO

USE RaceDayDB;
GO

/* Drop tables in FK-safe order if re-running the script */
IF OBJECT_ID('dbo.Results', 'U') IS NOT NULL DROP TABLE dbo.Results;
IF OBJECT_ID('dbo.Enrolments', 'U') IS NOT NULL DROP TABLE dbo.Enrolments;
IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL DROP TABLE dbo.Categories;
IF OBJECT_ID('dbo.Events', 'U') IS NOT NULL DROP TABLE dbo.Events;
IF OBJECT_ID('dbo.UserProfiles', 'U') IS NOT NULL DROP TABLE dbo.UserProfiles;
IF OBJECT_ID('dbo.Users', 'U') IS NOT NULL DROP TABLE dbo.Users;
GO

/* ============================================================
   1. Users
   ============================================================ */
CREATE TABLE dbo.Users (
    UserId          INT IDENTITY(1,1)       NOT NULL,
```

No issues found

SQLQuery1....mudau (64))*

```sql
28  CREATE TABLE dbo.Users (
29      UserId          INT IDENTITY(1,1)       NOT NULL,
30      FullName        NVARCHAR(100)           NOT NULL,
31      Email           NVARCHAR(150)           NOT NULL,
32      PasswordHash    NVARCHAR(255)           NOT NULL,
33      Role            NVARCHAR(20)            NOT NULL DEFAULT 'Participant',
34      CreatedAt       DATETIME                NOT NULL DEFAULT GETDATE(),
35      CONSTRAINT PK_Users PRIMARY KEY (UserId),
36      CONSTRAINT UQ_Users_Email UNIQUE (Email),
37      CONSTRAINT CK_Users_Role CHECK (Role IN ('Organiser', 'Participant'))
38  );
39  GO
40
41  /* ============================================================
42     2. UserProfiles  (1:1 with Users)
43     ============================================================ */
44  CREATE TABLE dbo.UserProfiles (
45      ProfileId       INT IDENTITY(1,1)       NOT NULL,
46      UserId          INT                     NOT NULL,
47      PhoneNumber     NVARCHAR(20)            NULL,
48      DateOfBirth     DATE                    NULL,
49      Bio             NVARCHAR(500)           NULL,
50      CONSTRAINT PK_UserProfiles PRIMARY KEY (ProfileId),
51      CONSTRAINT UQ_UserProfiles_UserId UNIQUE (UserId),
52      CONSTRAINT FK_UserProfiles_Users FOREIGN KEY (UserId)
53          REFERENCES dbo.Users (UserId) ON DELETE CASCADE
54  );
55  GO
56
```

100 % No issues found Ln: 1,


SQLQuery1....mudau (64))*
GO
/* ==
3. Events (1:M from Users as Organiser)
== */
CREATE TABLE dbo.Events (
EventId INT IDENTITY(1,1) NOT NULL,
OrganiserId INT NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(1000) NULL,
EventDate DATE NOT NULL,
Location NVARCHAR(200) NOT NULL,
Distance DECIMAL(6,2) NOT NULL,
EventType NVARCHAR(20) NOT NULL,
CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
CONSTRAINT PK_Events PRIMARY KEY (EventId),
CONSTRAINT FK_Events_Organiser FOREIGN KEY (OrganiserId)
REFERENCES dbo.Users (UserId),
CONSTRAINT CK_Events_EventType CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
CONSTRAINT CK_Events_Distance CHECK (Distance > 0)
);
GO
/* ==
4. Categories (1:M from Events)
== */
CREATE TABLE dbo.Categories (
CategoryId INT IDENTITY(1,1) NOT NULL,
EventId INT NOT NULL,
Name NVARCHAR(100) NOT NULL,
MinAge INT NULL,
MaxAge INT NULL,
CategoryDistance DECIMAL(6,2) NULL,
CONSTRAINT PK_Categories PRIMARY KEY (CategoryId),
CONSTRAINT FK_Categories_Events FOREIGN KEY (EventId)
REFERENCES dbo.Events (EventId) ON DELETE CASCADE
);
GO
/* ==
5. Enrolments (M:1 Users-as-Participant, M:1 Events, M:1 Categories)
== */
CREATE TABLE dbo.Enrolments (
EnrolmentId INT IDENTITY(1,1) NOT NULL,
ParticipantId INT NOT NULL,
EventId INT NOT NULL,
CategoryId INT NOT NULL,
EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),
CONSTRAINT PK_Enrolments PRIMARY KEY (EnrolmentId),
CONSTRAINT FK_Enrolments_Participant FOREIGN KEY (ParticipantId)
REFERENCES dbo.Users (UserId),
CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventId)
REFERENCES dbo.Events (EventId),
CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId)
REFERENCES dbo.Categories (CategoryId),
CONSTRAINT UQ_Enrolments_Participant_Event UNIQUE (ParticipantId, EventId)
);
GO
50 %
No issues found
Messages
50 %
No issues found

```
GO

/* ============================================================
   6. Results  (1:1 with Enrolments)
   ============================================================ */
CREATE TABLE dbo.Results (
    ResultId            INT IDENTITY(1,1)  NOT NULL,
    EnrolmentId         INT                NOT NULL,
    FinishTime          TIME               NULL,
    FinishingPosition   INT                NULL,
    CapturedAt          DATETIME           NOT NULL DEFAULT GETDATE(),
    CONSTRAINT PK_Results PRIMARY KEY (ResultId),
    CONSTRAINT UQ_Results_EnrolmentId UNIQUE (EnrolmentId),
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId)
        REFERENCES dbo.Enrolments (EnrolmentId) ON DELETE CASCADE
);
GO

/* ============================================================
   SEED DATA
   Note: PasswordHash values below are placeholder bcrypt-style
   strings for seed/demo purposes only - real hashes are produced
   by the API's registration endpoint, not typed here directly.
   ============================================================ */

-- 2 Organisers + 2 Participants
INSERT INTO dbo.Users (FullName, Email, PasswordHash, Role) VALUES
('Sarah Nkosi',     'sarah.nkosi@raceday.co.za',   '$2a$11$examplehash.organiser.one', 'Organiser'),
('David Botha',     'david.botha@raceday.co.za',   '$2a$11$examplehash.organiser.two', 'Organiser'),
('Lindiwe Dube',    'lindiwe.dube@example.com',    '$2a$11$examplehash.participant.a', 'Participant'),
('Michael Reddy',   'michael.reddy@example.com',   '$2a$11$examplehash.participant.b', 'Participant');
GO

INSERT INTO dbo.UserProfiles (UserId, PhoneNumber, DateOfBirth, Bio) VALUES
(1, '0821234567', '1985-03-12', 'Event organiser specialising in road running.'),
(2, '0837654321', '1979-11-02', 'Organiser for cycling and multi-sport events.'),
(3, '0715558899', '1998-06-21', 'Keen 10km and half-marathon runner.'),
(4, '0723334455', '1990-01-15', 'Weekend cyclist training for first century ride.');
GO

-- 3 Events (owned by the 2 organisers)
INSERT INTO dbo.Events (OrganiserId, Name, Description, EventDate, Location, Distance, EventType) VALUES
(1, 'Johannesburg City Run',   'Annual road running event through the city centre.', '2026-09-12', 'Johannesburg, Gauteng', 21.10, 'Run'),
(1, 'Sunrise Fun Walk',        'Community walk supporting local charities.',         '2026-08-30', 'Pretoria, Gauteng',     5.00, 'Walk'),
(2, 'Highveld Cycle Challenge','Road cycling race across the Highveld region.',      '2026-10-04', 'Bronkhorstspruit, Gauteng', 90.00, 'Cycle');
GO

-- Categories per event
INSERT INTO dbo.Categories (EventId, Name, MinAge, MaxAge, CategoryDistance) VALUES
(1, 'Senior 21km',   20, 39, 21.10),
(1, 'Veteran 21km',  40, 99, 21.10),
(1, 'Under 20',      13, 19, 21.10),
(2, '5km Walk',       0, 99, 5.00),
(3, '90km Road Race', 18, 99, 90.00),
(3, '45km Fun Ride',  14, 99, 45.00);
GO
```

50 % ⌄ ✓ No issues found

Messages

SQLQuery1....mudau (64))*

```
('Sarah Nkosi',     'sarah.nkosi@raceday.co.za',   '$2a$11$examplehash.organiser.one', 'Organiser'),
('David Botha',     'david.botha@raceday.co.za',   '$2a$11$examplehash.organiser.two', 'Organiser'),
('Lindiwe Dube',    'lindiwe.dube@example.com',    '$2a$11$examplehash.participant.a', 'Participant'),
('Michael Reddy',   'michael.reddy@example.com',   '$2a$11$examplehash.participant.b', 'Participant');
GO

INSERT INTO dbo.UserProfiles (UserId, PhoneNumber, DateOfBirth, Bio) VALUES
(1, '0821234567', '1985-03-12', 'Event organiser specialising in road running.'),
(2, '0837654321', '1979-11-02', 'Organiser for cycling and multi-sport events.'),
(3, '07155588899', '1998-06-21', 'Keen 10km and half-marathon runner.'),
(4, '07233344455', '1990-01-15', 'Weekend cyclist training for first century ride.');
GO

-- 3 Events (owned by the 2 organisers)
INSERT INTO dbo.Events (OrganiserId, Name, Description, EventDate, Location, Distance, EventType) VALUES
(1, 'Johannesburg City Run',    'Annual road running event through the city centre.', '2026-09-12', 'Johannesburg, Gauteng', 21.10, 'Run'),
(1, 'Sunrise Fun Walk',         'Community walk supporting local charities.',         '2026-08-30', 'Pretoria, Gauteng',     5.00, 'Walk'),
(2, 'Highveld Cycle Challenge','Road cycling race across the Highveld region.',      '2026-10-04', 'Bronkhorstspruit, Gauteng', 90.00, 'Cycle');
GO

-- Categories per event
INSERT INTO dbo.Categories (EventId, Name, MinAge, MaxAge, CategoryDistance) VALUES
(1, 'Senior 21km',    20, 39, 21.10),
(1, 'Veteran 21km',   40, 99, 21.10),
(1, 'Under 20',       13, 19, 21.10),
(2, '5km Walk',        0, 99, 5.00),
(3, '90km Road Race', 18, 99, 90.00),
(3, '45km Fun Ride',  14, 99, 45.00);
GO

-- Sample enrolments
INSERT INTO dbo.Enrolments (ParticipantId, EventId, CategoryId) VALUES
(3, 1, 1),   -- Lindiwe enters Johannesburg City Run, Senior 21km
(4, 1, 2),   -- Michael enters Johannesburg City Run, Veteran 21km
(3, 3, 5),   -- Lindiwe enters Highveld Cycle Challenge, 90km Road Race
(4, 2, 4);   -- Michael enters Sunrise Fun Walk, 5km Walk
GO

-- Sample results for a couple of completed enrolments
INSERT INTO dbo.Results (EnrolmentId, FinishTime, FinishingPosition) VALUES
(1, '01:45:32', 12),
(2, '01:58:10', 27);
GO
```

100 % No issues found

Messages | Client Statistics

	Trial 1	Average
Query Profile Statistics		
Number of INSERT, DELETE and UPDATE statements	6	6.0000
Rows affected by INSERT, DELETE, or UPDATE stateme...	23	23.0000
Number of SELECT statements	0	0.0000
Rows returned by SELECT statements	0	0.0000
Number of transactions	6	6.0000
Network Statistics		
Number of server roundtrips	15	15.0000
TDS packets sent from client	15	15.0000
TDS packets received from server	15	15.0000
Bytes sent from client	17278	17278.0000
Bytes received from server	735	735.0000
Time Statistics		
Client processing time	4	4.0000
Total execution time	328	328.0000
Wait time on server replies	324	324.0000

Query executed successfully. | IMANI\SQLEXPRESS (16.0 RTM) | IMANI\mudau (64) | RaceDayDB | 00

API Endpoint Plan

This plan covers Authentication, User Profile, Events, Categories, Event Enrolments, and Results, as required by the Part 2 Functional Requirements. Role-based access is enforced on every non-public endpoint; session management maintains the authenticated user's identity and role for subsequent requests.

Authentication

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as either an Organiser or a Participant.	None (public)	{ fullName, email, password, role }	201 Created - user record created (no password returned) 400 Bad Request - invalid/missing fields 409 Conflict - email already registered
POST	/api/auth/login	Authenticates a user and establishes their session, recording identity and role.	None (public)	{ email, password }	200 OK - session established, user id and role returned 401 Unauthorized - invalid credentials
POST	/api/auth/logout	Ends the current authenticated session.	Any (logged in)	None	200 OK - session ended 401 Unauthorized - no active session

User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me	Returns the logged-in user's own account and profile details.	Any (logged in)	None	200 OK - user + profile data 401 Unauthorized - not logged in
PUT	/api/users/me	Updates the logged-in user's own profile information.	Any (logged in)	{ fullName, phoneNumber, dateOfBirth, bio }	200 OK - updated profile returned 400 Bad Request - invalid fields 401 Unauthorized - not logged in

Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Lists all events. Visible to both roles for browsing.	Any (logged in)	None	200 OK - array of events
GET	/api/events/{id}	Retrieves the details of a single event.	Any (logged in)	None	200 OK - event details 404 Not Found - event does not exist
POST	/api/events	Creates a new event owned by the logged-in Organiser.	Organiser	{ name, description, eventDate, location, distance, eventType }	201 Created - new event record 400 Bad Request - invalid fields 403 Forbidden - caller is a Participant
PUT	/api/events/{id}	Updates an event owned by the logged-in Organiser.	Organiser	{ name, description, eventDate, location, distance, eventType }	200 OK - updated event 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist
DELETE	/api/events/{id}	Deletes an event owned by the logged-in Organiser.	Organiser	None	204 No Content - event deleted 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist

Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Lists all categories available for a given event.	Any (logged in)	None	200 OK - array of categories 404 Not Found - event does not exist
POST	/api/events/{eventId}/categories	Defines a new age or distance	Organiser	{ name, minAge, maxAge,	201 Created - new category record 403

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		category for an event owned by the logged-in Organiser.		categoryDistance }	Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist
PUT	/api/categories/{id}	Updates a category belonging to an event owned by the logged-in Organiser.	Organiser	{ name, minAge, maxAge, categoryDistance }	200 OK - updated category 403 Forbidden - not the owning organiser or a Participant 404 Not Found - category does not exist
DELETE	/api/categories/{id}	Deletes a category belonging to an event owned by the logged-in Organiser.	Organiser	None	204 No Content - category deleted 403 Forbidden - not the owning organiser or a Participant 404 Not Found - category does not exist

Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/events/{eventId}/enrolments	Enters the logged-in Participant into the event under a selected category.	Participant	{ categoryId }	201 Created - enrolment record linking participant, event and category 400 Bad Request - category does not belong to the event 403 Forbidden - caller

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					is an Organiser 404 Not Found - event or category does not exist 409 Conflict - participant already enrolled in this event
GET	/api/events/{eventId}/enrolments	Lists all enrolments for an event owned by the logged-in Organiser.	Organiser	None	200 OK - array of enrolments with participant and category details 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist
GET	/api/users/me/enrolments	Lists the logged-in Participant's own enrolments across all events.	Participant	None	200 OK - array of the participant's enrolments

Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{enrolmentId}/results	Captures the finish time and finishing position for a participant's	Organiser	{ finishTime, finishingPosition }	201 Created - result record created 403 Forbidden - not the event's organiser or a Participant 404 Not Found -

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		enrolment, by the event's Organiser.			enrolment does not exist 409 Conflict - result already captured for this enrolment
PUT	/api/enrolments/{enrolmentId}/results	Updates a previously captured result for a participant's enrolment.	Organiser	{ finishTime, finishingPosition }	200 OK - updated result 403 Forbidden - not the event's organiser or a Participant 404 Not Found - result does not exist
GET	/api/users/me/results	Lists the logged-in Participant's own results across all events entered.	Participant	None	200 OK - array of results with event and category context
GET	/api/events/{eventId}/results	Lists all results for an event owned by the logged-in Organiser.	Organiser	None	200 OK - array of results for the event 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist

ReadME - Part 1 Planning Documentation

Contents

- **erd.png / erd.pdf** - Entity Relationship Diagram for the full RaceDay data model.
- **schema.sql** - SQL Server script that creates every table in the ERD, with all primary keys, foreign keys, and constraints, plus seed data (2 Organisers, 2 Participants, 3 Events, categories per event, and sample enrolments/results).
- **api_endpoint_plan.md** - Full API endpoint plan covering Authentication, User Profile, Events, Categories, Event Enrolments, and Results.

Data model summary (6 entities)

1. **Users** - both Organisers and Participants; role stored on the user record.
2. **UserProfiles** - 1:1 with Users; holds extended profile fields separate from login/auth data.
3. **Events** - 1:M from Users (Organiser); each event has a name, description, date, location, distance, and event type (Run, Walk, Cycle).
4. **Categories** - 1:M from Events; each event defines its own age/distance categories.
5. **Enrolments** - the link entity between a Participant (Users), an Event, and the Category they selected. A participant may enrol in a given event only once (enforced by a unique constraint on ParticipantId + EventId).
6. **Results** - 1:1 with Enrolments; captures finish time and finishing position once an event has taken place.

Consistency note

schema.sql matches erd.png exactly - the same six entities, the same primary/foreign keys, and the same cardinalities (1:1 for Users-UserProfiles and Enrolments-Results; 1:M everywhere else). No deliberate deviations were introduced between the diagram and the script.